THỰC HÀNH 6: COLLECTIONS FRAMEWORK & STREAM API

🎯 MỤC TIÊU BÀI HỌC

Sau khi hoàn thành bài thực hành này, bạn sẽ:

1. ✅ **Sử dụng thành thạo** Collections (List, Set, Map)
2. ✅ **Chọn đúng** cấu trúc dữ liệu cho từng bài toán
3. ✅ **Hiểu Generics** và Type Safety
4. ✅ **Viết Lambda Expressions** thành thạo
5. ✅ **Xử lý dữ liệu** với Stream API
6. ✅ **Chuyển đổi** từ Imperative sang Declarative

⏱️ **Thời gian ước tính:** 7-10 giờ

📋 **Kiến thức cần có trước:**

- Đã hoàn thành Thực hành 5
- Đã đọc Chương 6: Collections & Stream API
- Hiểu về Wrapper Classes (Integer, Double, ...)

🧪 PHẦN 1: HƯỚNG DẪN CHI TIẾT (STEP-BY-STEP)

LAB 6.1: Cuộc Đua Tốc Độ (List vs Set vs Map)

Mục tiêu: Chứng minh sức mạnh tìm kiếm của HashMap

Bước 1: Benchmark Tốc Độ

Tạo file: `Benchmark.java`

```java
package com.bootcamp.lab6;

import java.util.*;

public class Benchmark {
    public static void main(String[] args) {
        final int LIMIT = 100000;  // 100 nghìn phần tử

        // Tạo List và Set
        List<Integer> list = new ArrayList<>();
        Set<Integer> set = new HashSet<>();
        Map<Integer, Integer> map = new HashMap<>();

        System.out.println("⏳ Đang nạp " + LIMIT + " phần tử...");
```

```java
        for (int i = 0; i < LIMIT; i++) {
            list.add(i);
            set.add(i);
            map.put(i, i * 2);  // Key = i, Value = i*2
        }

        int searchValue = LIMIT - 1;  // Tìm phần tử cuối cùng (tệ nhất)

        // Test 1: Tìm trong List (O(n))
        System.out.println("\n🚀 Bắt đầu đo LIST...");
        long start = System.nanoTime();
        boolean foundInList = list.contains(searchValue);
        long end = System.nanoTime();
        System.out.println("List Time: " + (end - start) + " ns");
        System.out.println("   Kết quả: " + foundInList);

        // Test 2: Tìm trong Set (O(1))
        System.out.println("\n🚀 Bắt đầu đo SET...");
        start = System.nanoTime();
        boolean foundInSet = set.contains(searchValue);
        end = System.nanoTime();
        System.out.println("Set Time : " + (end - start) + " ns");
        System.out.println("   Kết quả: " + foundInSet);

        // Test 3: Tìm trong Map (O(1))
        System.out.println("\n🚀 Bắt đầu đo MAP...");
        start = System.nanoTime();
        boolean foundInMap = map.containsKey(searchValue);
        end = System.nanoTime();
        System.out.println("Map Time : " + (end - start) + " ns");
        System.out.println("   Kết quả: " + foundInMap);

        // Kết quả: Set và Map nhanh hơn List cả nghìn lần!
    }
}
```

Kết quả mẫu:

```
List Time: 1234567 ns  (Chậm - phải duyệt cả list)
Set Time : 1234 ns     (Nhanh - tìm trực tiếp)
Map Time : 1456 ns     (Nhanh - tìm trực tiếp)
```

Bài học: Nếu cần tìm kiếm nhiều → Dùng Set hoặc Map, không dùng List!

LAB 6.2: Dây Chuyền Xử Lý Dữ Liệu (Stream API)

Mục tiêu: Xử lý dữ liệu theo kiểu Functional Programming

Bước 1: Chuẩn Bị Data

Tạo file: `Employee.java`

```java
package com.bootcamp.lab6;

public class Employee {
    private String name;
    private String department;
    private double salary;

    public Employee(String name, String department, double salary) {
        this.name = name;
        this.department = department;
        this.salary = salary;
    }

    public String getName() { return name; }
    public String getDepartment() { return department; }
    public double getSalary() { return salary; }

    @Override
    public String toString() {
        return name + " (" + department + ") - " + salary + " VND";
    }
}
```

Bước 2: Stream Pipeline

Tạo file: `StreamDemo.java`

```java
package com.bootcamp.lab6;

import java.util.*;
import java.util.stream.Collectors;

public class StreamDemo {
    public static void main(String[] args) {
        // Tạo danh sách nhân viên
        List<Employee> staff = Arrays.asList(
            new Employee("Nguyễn Văn A", "IT", 20000000),
            new Employee("Trần Thị B", "Sale", 15000000),
            new Employee("Lê Văn C", "IT", 18000000),
            new Employee("Phạm Thị D", "IT", 30000000),
            new Employee("Hoàng Văn E", "HR", 12000000)
        );

        System.out.println("=== PHÂN TÍCH NHÂN VIÊN ===\n");

        // Yêu cầu: Lọc IT, lương > 15 triệu, tăng lương 10%,
        // sắp xếp giảm dần, lấy top 3, in tên
```

```
        List<String> result = staff.stream()  // Mở băng chuyền
            // Bước 1: Lọc (Filter)
            .filter(e -> e.getDepartment().equals("IT"))
            .filter(e -> e.getSalary() > 15000000)

            // Bước 2: Biến đổi (Map) - Tăng lương 10%
            .map(e -> {
                double newSalary = e.getSalary() * 1.1;
                return new Employee(e.getName(), e.getDepartment(),
newSalary);
            })

            // Bước 3: Sắp xếp (Sort) - Giảm dần theo lương
            .sorted((e1, e2) -> Double.compare(e2.getSalary(),
e1.getSalary()))

            // Bước 4: Cắt gọn (Limit) - Lấy top 3
            .limit(3)

            // Bước 5: Ép khuôn đầu ra (Map to String)
            .map(e -> "🎉 " + e.getName() + " - Lương mới: " +
                      String.format("%.0f", e.getSalary()) + " VND")

            // Bước 6: Đóng gói (Collect)
            .collect(Collectors.toList());

        // In ra kết quả
        System.out.println("Top 3 nhân viên IT sau khi tăng lương:\n");
        result.forEach(System.out::println);

        // Thống kê khác
        System.out.println("\n=== THỐNG KÊ ===\n");

        // Tính tổng lương IT
        double totalITSalary = staff.stream()
            .filter(e -> e.getDepartment().equals("IT"))
            .mapToDouble(Employee::getSalary)
            .sum();
        System.out.printf("Tổng lương phòng IT: %.0f VND\n",
totalITSalary);

        // Đếm số nhân viên IT
        long itCount = staff.stream()
            .filter(e -> e.getDepartment().equals("IT"))
            .count();
        System.out.println("Số nhân viên IT: " + itCount);

        // Tìm nhân viên lương cao nhất
        Optional<Employee> maxSalary = staff.stream()
            .max((e1, e2) -> Double.compare(e1.getSalary(),
e2.getSalary()));

        maxSalary.ifPresent(e ->
            System.out.println("Nhân viên lương cao nhất: " + e)
```

```
            );
        }
    }
```

Giải thích:

1. **Stream Pipeline:**
   - `stream()`: Mở băng chuyền
   - `filter()`: Lọc (giữ lại thỏa điều kiện)
   - `map()`: Biến đổi (transform)
   - `sorted()`: Sắp xếp
   - `limit()`: Giới hạn số lượng
   - `collect()`: Thu thập kết quả
2. **Method Reference:**
   - `Employee::getSalary` tương đương `e -> e.getSalary()`
   - `System.out::println` tương đương `s -> System.out.println(s)`
3. **Optional:**
   - `max()` trả về `Optional<Employee>`
   - Có thể không có (nếu list rỗng)
   - `ifPresent()`: Chỉ chạy nếu có giá trị

LAB 6.3: Từ Điển Anh-Việt (HashMap)

Mục tiêu: Sử dụng HashMap để tra cứu nhanh

Bước 1: Tạo Dictionary

Tạo file: `Dictionary.java`

```
package com.bootcamp.lab6;

import java.util.*;

public class Dictionary {
    private Map<String, String> dict;

    public Dictionary() {
        dict = new HashMap<>();
        initializeDictionary();
    }

    private void initializeDictionary() {
        dict.put("hello", "Xin chào");
        dict.put("world", "Thế giới");
        dict.put("java", "Ngôn ngữ lập trình Java");
```

```
        dict.put("computer", "Máy tính");
        dict.put("programming", "Lập trình");
        dict.put("code", "Mã code");
        dict.put("algorithm", "Thuật toán");
        dict.put("data", "Dữ liệu");
    }

    public String translate(String englishWord) {
        // getOrDefault: Trả về giá trị hoặc giá trị mặc định
        return dict.getOrDefault(englishWord.toLowerCase(),
                                 "❌ Không tìm thấy từ: " + englishWord);
    }

    public void addWord(String english, String vietnamese) {
        dict.put(english.toLowerCase(), vietnamese);
        System.out.println("✅ Đã thêm từ: " + english + " -> " +
vietnamese);
    }

    public void displayAll() {
        System.out.println("\n=== TỪ ĐIỂN ===");
        dict.forEach((key, value) ->
            System.out.println(key + " -> " + value)
        );
    }
}
```

Bước 2: Test Dictionary

Tạo file: `DictionaryApp.java`

```
package com.bootcamp.lab6;

import java.util.Scanner;

public class DictionaryApp {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Dictionary dict = new Dictionary();

        System.out.println("=== TỪ ĐIỂN ANH-VIỆT ===\n");

        while (true) {
            System.out.print("Nhập từ tiếng Anh (hoặc 'quit' để thoát):
");
            String word = sc.nextLine().trim();

            if (word.equalsIgnoreCase("quit")) {
                break;
            }
```

```
            if (word.isEmpty()) {
                continue;
            }

            String meaning = dict.translate(word);
            System.out.println("   " + meaning + "\n");
        }

        sc.close();
    }
}
```

📝 PHẦN 2: BÀI TẬP CÓ HƯỚNG DẪN

Bài 6.1: Đếm Số Lần Xuất Hiện (Word Count)

Đề bài: Đếm số lần mỗi từ xuất hiện trong câu.

Ví dụ: "Java is cool and Java is fun"

- Output: Java=2, is=2, cool=1, and=1, fun=1

Hướng Dẫn

```java
package com.bootcamp.lab6;

import java.util.*;

public class WordCount {
    public static void main(String[] args) {
        String text = "Java is cool and Java is fun";

        // Tách thành mảng từ
        String[] words = text.split(" ");

        // Dùng Map để đếm
        Map<String, Integer> wordCount = new HashMap<>();

        // Cách 1: Dùng vòng lặp (Imperative)
        for (String word : words) {
            if (wordCount.containsKey(word)) {
                // Từ đã có → Tăng count
                wordCount.put(word, wordCount.get(word) + 1);
            } else {
                // Từ mới → Thêm vào với count = 1
                wordCount.put(word, 1);
            }
        }

        // Cách 2: Dùng Stream API (Declarative) - Ngắn gọn hơn!
```

```
        Map<String, Long> wordCount2 = Arrays.stream(words)
            .collect(Collectors.groupingBy(
                word -> word,  // Nhóm theo từ
                Collectors.counting()  // Đếm số lần
            ));

        // Hiển thị kết quả
        System.out.println("=== KẾT QUẢ ===");
        wordCount.forEach((word, count) ->
            System.out.println(word + " = " + count)
        );
    }
}
```

Bài 6.2: Lọc và Sắp Xếp Sinh Viên

Đề bài: Cho danh sách sinh viên, dùng Stream API để:

- Lọc sinh viên có GPA >= 3.5
- Sắp xếp theo GPA giảm dần
- Lấy top 5
- In danh sách

Hướng Dẫn

```java
package com.bootcamp.lab6;

import java.util.*;
import java.util.stream.Collectors;

class Student {
    private String name;
    private double gpa;

    public Student(String name, double gpa) {
        this.name = name;
        this.gpa = gpa;
    }

    public String getName() { return name; }
    public double getGpa() { return gpa; }

    @Override
    public String toString() {
        return name + " - GPA: " + gpa;
    }
}

public class StudentFilter {
    public static void main(String[] args) {
```

```
        List<Student> students = Arrays.asList(
            new Student("Nguyễn Văn A", 3.8),
            new Student("Trần Thị B", 3.2),
            new Student("Lê Văn C", 3.9),
            new Student("Phạm Thị D", 3.6),
            new Student("Hoàng Văn E", 3.4),
            new Student("Vũ Thị F", 3.7),
            new Student("Đỗ Văn G", 3.5)
        );

        List<Student> topStudents = students.stream()
            .filter(s -> s.getGpa() >= 3.5)
            .sorted((s1, s2) -> Double.compare(s2.getGpa(), s1.getGpa()))
            .limit(5)
            .collect(Collectors.toList());

        System.out.println("=== TOP 5 SINH VIÊN (GPA >= 3.5) ===\n");
        topStudents.forEach(System.out::println);
    }
}
```

🎯 PHẦN 3: BÀI TẬP TỰ LUYỆN (KHÔNG CÓ HƯỚNG DẪN)

Bài 6.1: Quản Lý Danh Sách với ArrayList

Đề bài: Tạo class `ListManager` quản lý danh sách số nguyên:

- Thêm số
- Xóa số
- Tìm số
- Hiển thị tất cả
- Tính tổng, trung bình
- Tìm số lớn nhất, nhỏ nhất

Yêu cầu:

- Dùng `ArrayList<Integer>`
- Menu cho user chọn

Bài 6.2: Loại Bỏ Duplicate với Set

Đề bài: Nhập danh sách tên sinh viên (có thể trùng). In ra danh sách không trùng.

Yêu cầu:

- Dùng `HashSet<String>`
- Giữ thứ tự chèn → Dùng `LinkedHashSet`

Bài 6.3: Thống Kê với Stream API

Đề bài: Cho danh sách sản phẩm (name, price, category). Dùng Stream API:

- Tính tổng giá trị
- Tìm sản phẩm đắt nhất
- Nhóm theo category
- Tính giá trung bình mỗi category

Bài 6.4: Lambda Expressions

Đề bài: Viết các Lambda expressions:

- Sắp xếp List theo độ dài
- Lọc List chỉ lấy số chẵn
- Chuyển List thành chữ hoa
- Tính tổng List

Bài 6.5: Word Count với Map

Đề bài: Đọc một đoạn văn bản, đếm số lần xuất hiện của mỗi từ:

- Dùng `HashMap<String, Integer>` để lưu từ → số lần
- Bỏ qua chữ hoa/thường (chuyển về lowercase)
- Loại bỏ dấu câu
- Hiển thị top 10 từ xuất hiện nhiều nhất

Gợi ý:

- Tách chuỗi thành mảng từ bằng `split(" ")`
- Dùng `map.getOrDefault(word, 0)` để đếm
- Sắp xếp Map theo value giảm dần

Bài 6.6: Filter và Transform với Stream

Đề bài: Cho danh sách `Student` (name, age, gpa):

- Lọc sinh viên có GPA >= 8.0
- Chuyển thành danh sách tên (String)
- Sắp xếp theo tên A-Z
- Hiển thị kết quả

Gợi ý:

- Dùng `filter()` để lọc
- Dùng `map()` để chuyển Student → String (tên)
- Dùng `sorted()` để sắp xếp
- Dùng `collect(Collectors.toList())` để thu thập

Bài 6.7: Grouping và Aggregation với Stream

Đề bài: Cho danh sách `Product` (name, price, category):

- Nhóm sản phẩm theo category
- Tính tổng giá trị mỗi category
- Tìm category có tổng giá trị cao nhất
- Tính giá trung bình mỗi category

Gợi ý:

- Dùng `Collectors.groupingBy()` để nhóm
- Dùng `Collectors.summingDouble()` để tính tổng
- Dùng `Collectors.averagingDouble()` để tính trung bình
- Dùng `max()` với Comparator để tìm category cao nhất

Bài 6.8: Set Operations (Giao, Hợp, Hiệu)

Đề bài: Cho 2 danh sách số nguyên:

- Tìm các số xuất hiện ở cả 2 danh sách (giao)
- Tìm tất cả số (hợp, không trùng)
- Tìm số chỉ có ở danh sách 1 (hiệu)

Gợi ý:

- Dùng `HashSet` để loại bỏ duplicate
- `retainAll()`: Giao
- `addAll()`: Hợp
- `removeAll()`: Hiệu
- Hoặc dùng Stream API với `filter()` và `contains()`

Bài 6.9: Complex Stream Pipeline

Đề bài: Cho danh sách `Order` (id, customer, items, total, date):

- Lọc đơn hàng trong tháng hiện tại
- Lọc đơn hàng có tổng > 500,000
- Nhóm theo khách hàng
- Tính tổng tiền mỗi khách hàng
- Sắp xếp theo tổng tiền giảm dần
- Lấy top 5 khách hàng

Gợi ý:

- Dùng `filter()` nhiều lần (có thể chain)
- Dùng `Collectors.groupingBy()` với `Collectors.summingDouble()`
- Dùng `sorted()` với `Comparator.reverseOrder()`
- Dùng `limit(5)` để lấy top 5
- Dùng `LocalDate` để xử lý ngày tháng

🏆 PHẦN 4: THỬ THÁCH NÂNG CAO

Challenge 1: Big Data Analysis

Đề bài: Xử lý file `transactions.csv` với 1 triệu dòng.

Format: `ID,Năm,ThànhPhố,GiáTrị`

Yêu cầu dùng Stream API:

1. Có bao nhiêu giao dịch ở "Hanoi" năm 2023?
2. Tìm giao dịch có giá trị lớn nhất
3. Liệt kê danh sách thành phố (không trùng)
4. Nhóm giao dịch theo năm
5. Tính tổng giá trị mỗi thành phố

Gợi ý:

- Dùng `Files.lines()` để đọc file
- Dùng `Collectors.groupingBy()`

Challenge 2: Product Manager với Stream

Đề bài: Tạo hệ thống quản lý sản phẩm:

1. **Class `Product`:**
   - Attributes: id, name, price, category, stock
2. **Class `ProductManager`:**
   - Sử dụng `List<Product>`
   - Tất cả methods dùng Stream API:
     - `findByCategory(String category)`
     - `findExpensiveProducts(double minPrice)`
     - `getAveragePrice()`
     - `getProductsByPriceRange(double min, double max)`
     - `getTopExpensiveProducts(int n)`
     - `groupByCategory()`: Trả về `Map<String, List<Product>>`
     - `getLowStockProducts(int threshold)`: Sản phẩm sắp hết hàng

Yêu cầu:

- Không dùng vòng lặp `for` (chỉ dùng Stream)
- Code Declarative

Challenge 3: Advanced Stream Operations

Đề bài: Cho danh sách đơn hàng (Order: id, customer, items, total):

1. Tìm đơn hàng có tổng tiền lớn nhất
2. Tính tổng doanh thu
3. Nhóm đơn hàng theo khách hàng
4. Tìm khách hàng mua nhiều nhất
5. Tính giá trị trung bình mỗi đơn hàng
6. Lọc đơn hàng > 1 triệu, sắp xếp giảm dần

Yêu cầu:

- Dùng Stream API hoàn toàn
- Sử dụng `Collectors` đầy đủ

📚 BẢNG GHI NHỚ (CHEAT SHEET)

Collections Selection

Cần gì?	Dùng gì?	Ví dụ
Có duplicate, có thứ tự	`ArrayList`	Danh sách sinh viên
Không duplicate	`HashSet`	Danh sách ID duy nhất
Key-Value	`HashMap`	Từ điển, Cache
Cần sorted	`TreeSet`, `TreeMap`	Leaderboard

Stream API Operations

Operation	Loại	Ví dụ
`filter()`	Intermediate	`.filter(x -> x > 10)`
`map()`	Intermediate	`.map(x -> x * 2)`
`sorted()`	Intermediate	`.sorted()`
`distinct()`	Intermediate	`.distinct()`
`limit()`	Intermediate	`.limit(5)`
`collect()`	Terminal	`.collect(Collectors.toList())`
`forEach()`	Terminal	`.forEach(System.out::println)`
`reduce()`	Terminal	`.reduce(0, Integer::sum)`

Lambda vs Method Reference

```
// Lambda
list.forEach(s -> System.out.println(s));

// Method Reference (ngắn gọn hơn)
list.forEach(System.out::println);
```

✅ CHECKLIST HOÀN THÀNH

- ☐ Đã sử dụng thành thạo List, Set, Map
- ☐ Đã chọn đúng cấu trúc dữ liệu cho bài toán
- ☐ Đã hiểu Generics và Type Safety
- ☐ Đã viết Lambda Expressions
- ☐ Đã xử lý dữ liệu với Stream API
- ☐ Đã chuyển đổi code từ Imperative sang Declarative
- ☐ Đã hoàn thành LAB 6.1 (Benchmark)
- ☐ Đã hoàn thành LAB 6.2 (Stream Demo)
- ☐ Đã hoàn thành LAB 6.3 (Dictionary)
- ☐ Đã làm ít nhất 2 bài tập có hướng dẫn
- ☐ Đã làm ít nhất 2 bài tập tự luyện
- ☐ Đã thử ít nhất 1 thử thách nâng cao

🎓 TỔNG KẾT

Sau bài thực hành này, bạn đã:

1. ✅ **Sử dụng Collections** (List, Set, Map) thành thạo
2. ✅ **Chọn đúng** cấu trúc dữ liệu
3. ✅ **Hiểu Generics** và Type Safety
4. ✅ **Viết Lambda Expressions**
5. ✅ **Xử lý dữ liệu** với Stream API
6. ✅ **Tư duy Functional Programming**

🎉 **Chúc mừng! Bạn đã hoàn thành 6 bài thực hành!**

Bây giờ bạn đã có nền tảng vững chắc để:

- Tham gia các dự án Java thực tế
- Học các framework (Spring, Hibernate)
- Phát triển ứng dụng web, mobile

Tiếp tục học tập và thực hành! 🚀

Chúc bạn học tốt! 🚀